

Fashion-MNIST 十分类实验报告

姓名：祝梓元 学号：U202314554 日期：2026 年 9 月 25 日

一、实验目的与任务

本实验在 Fashion-MNIST 数据集上完成十分类任务，围绕「线性模型的表达能力边界」与「训练过程的诊断方法」两条主线，包含以下四项任务：

- 线性基线**：训练线性 Softmax 分类器，记录训练与验证曲线，给出测试准确率、混淆矩阵、逐类召回率与典型错误可视化。
- 反向传播**：仅使用 NumPy 在固定 mini-batch 上实现单隐藏层 MLP 的前向与反向传播，并用中心差分完成梯度检查。
- 受控比较**：使用 PyTorch 比较线性 Softmax、两个线性层但中间无激活、以及加入 ReLU 的 MLP 三种模型，后两者使用相同隐藏层宽度，其余训练条件保持一致。
- 训练诊断**：选择一个实际出现的训练问题，先提出原因判断，再只改变一个关键因素进行验证。

二、实验环境与数据

2.1 软硬件环境

表 1 实验环境

项目	配置
操作系统	Windows 11
Python	3.13.3
NumPy	2.1.1
PyTorch / torchvision	2.12.0+cpu / 0.27.0+cpu
Matplotlib	3.10.7
计算设备	CPU（无 CUDA，全部实验在 CPU 上完成）

2.2 数据集与划分

Fashion-MNIST 包含 10 类服饰灰度图，每张 28×28 。官方训练集 60 000 张、测试集 10 000 张。

按要求将官方训练集固定划分为 50 000 训练 + 10 000 验证，官方测试集 10 000 张仅用于最终评价，不参与任何训练或超参选择。划分由随机种子 42 生成，索引落盘保存，四个实验共用同一份划分——这是后文受控比较能够成立的前提。

归一化使用训练集的均值与标准差（mean = 0.2860，std = 0.3531），三个模型共用同一组参数，避免「每个模型各归一化一套」这种隐性变量。

2.3 训练配置

表 2 统一训练配置（三个模型完全一致）

配置项	取值
优化器	SGD, momentum = 0.9
学习率	0.01
批大小	128
训练轮数	30
权重衰减	0（避免引入额外变量）
隐藏层宽度	256（模型②③共用）
随机种子	42

学习率 0.01 并非随意指定，而是在诊断实验中实测选定的安全值，详见第七节。

三、实验原理

3.1 Softmax 线性分类器

线性分类器对输入 $x \in R^{784}$ 计算各类得分（logits） $z = Wx + b$ ，再经 Softmax 归一化为概率：

$$p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

(1)

训练目标是最大化真实类别 y 的预测概率，等价于最小化交叉熵损失：

$$L = - \frac{1}{N} \sum_i \log p_{y_i}$$

(2)

Softmax 与交叉熵合并求导后形式非常简洁，这一点在 3.2 节的反向传播中会直接用到：

$$\frac{\partial L}{\partial z_k} = \frac{p_k - 1[k=y]}{N}$$

(3)

3.2 单隐藏层 MLP 与反向传播

单隐藏层 MLP 在两层线性变换之间插入 ReLU 激活：

$$Z_1 = XW_1 + b_1 \quad (4)$$

$$A_1 = \max(0, Z_1)$$

$$Z_2 = A_1W_2 + b_2$$

$$P = \text{softmax}(Z_2)$$

其中 $W_1 \in R^{784 \times 256}$, $W_2 \in R^{256 \times 10}$ 。反向传播按链式法则自上而下逐层回传，逐步推导如下：

$$dZ_2 = \frac{P - \text{onehot}(y)}{N} \Rightarrow dW_2 = A_1^T dZ_2, \quad db_2 = \Sigma dZ_2 \quad (5)$$

$$dA_1 = dZ_2 W_2^T \Rightarrow dZ_1 = dA_1 \odot 1[Z_1 > 0] \Rightarrow dW_1 = X^T dZ_1, \quad db_1 = \Sigma dZ_1 \quad (6)$$

式 (6) 中 $1[Z_1 > 0]$ 是 ReLU 的导数：正值处梯度原样通过，负值处截断为 0。全部实现使用 float64，原因见 3.3 节。

3.3 中心差分梯度检查

同一个梯度可以用两条完全独立的路径计算：**解析梯度**由手写反向传播按链式法则推出，速度快但可能推导或编码出错；**数值梯度**由差商近似，速度慢但只依赖前向传播，几乎不可能写错。梯度检查就是用后者检验前者。

数值微分有三种取法，精度差异极大：

表 3 三种差商近似的截断误差

方法	表达式	误差量级
前向差分	$[f(x+h) - f(x)] / h$	$O(h)$
后向差分	$[f(x) - f(x-h)] / h$	$O(h)$
中心差分	$[f(x+h) - f(x-h)] / 2h$	$O(h^2)$

中心差分精度高出两个数量级，原因可由泰勒展开看出：

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(x) + \dots$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2} f''(x) - \frac{h^3}{6} f'''(x) + \dots$$

两式相减， h^2 项因符号相同而被消去，只剩 h^3 项，除以 $2h$ 后误差主项为 $-(h^2/6)f'''(x)$ ；前向差分无法消去 h^2 项，误差主项为 $(h/2)f''(x)$ ，量级 $O(h)$ 。

取 $h = \varepsilon = 10^{-4}$ 时，中心差分的截断误差按 h^2 缩小到 10^{-9} 量级，前向差分仍有 10^{-5} 量级，两者相差约四个数量级。

但步长并非越小越好。中心差分是「两个相近的数相减」，会遭受灾难性抵消：计算 $f(x \pm h)$ 时各带有约 $\varepsilon/|L|$ 的舍入误差（ ε 为机器精度），二者相减后除以 $2h$ ，舍入误差被放大为

$$\delta_{round} \approx \frac{\varepsilon/L}{2h}$$

截断误差随 h 减小而减小，舍入误差却随 h 减小而增大，因此存在一个最优步长。令 $(h^2/6)/f''' = \varepsilon/L/(2h)$ 可得

$$h^* \approx \left(\frac{3\varepsilon/L}{|f''|} \right)^{1/3}$$

为什么必须用 float64。代入 $\varepsilon \approx 2.2 \times 10^{-16}$ 、 $|L| \approx 3.9$ 、 $|f''| \sim O(1)$ ，最优步长约 1.4×10^{-5} ；此时舍入误差 $\varepsilon/L/(2h) \approx 4.4 \times 10^{-12}$ ，远小于截断误差，梯度检查才有意义。若改用 float32 ($\varepsilon \approx 1.2 \times 10^{-7}$)，同一 $h = 10^{-4}$ 下舍入误差升到 2.3×10^{-3} ，已经比 10^{-6} 的判据本身还高三个数量级，测出的「误差」将完全是浮点噪声，梯度检查完全失去意义。

解析梯度与数值梯度的差异用相对误差衡量：

$$rel = \frac{|g_{ana} - g_{num}|}{\max(10^{-8}, |g_{ana}| + |g_{num}|)} \quad (7)$$

分母设置下限 10^{-8} 是为了避免梯度本身接近 0 时分母趋零、产生虚假的巨大相对误差。判据取 $rel < 10^{-6}$ 。

3.4 评价指标

混淆矩阵： 10×10 矩阵，约定行为真实类别、列为预测类别， $cm[i][j]$ 表示真实为 i 而被预测为 j 的样本数。对角线为分类正确的样本，非对角线元素直接暴露「哪几类之间容易互相混淆」。

逐类召回率： 衡量某一类的样本被找回的比例，

$$R_i = \frac{cm[i][i]}{\sum_j cm[i][j]} \quad (8)$$

即对角线元素除以该行总和。召回率关注「漏检」，与之相对的精确率则以列总和为分母，关注「误报」。

四、实验一：反向传播与梯度检查

4.1 实验方法

按式 (4) - (6) 用纯 NumPy 实现单隐藏层 MLP 的前向与反向传播，不使用任何自动求导。梯度检查在一个固定的 mini-batch (64 个样本, float64) 上进行——固定是必需的，否则两次前向传播的损失不可比。当前参数下该 batch 的交叉熵损失为 3.918401。

受计算量限制无法逐参数检查：MLP 共有 203 530 个参数，每个参数需要 2 次前向传播，总计约 40 万次前向，不具可行性。因此采用分层抽样，对 W_1, b_1, W_2, b_2 各抽 20 个参数，共检查 80 个，保证四组参数都被覆盖。

4.2 ReLU 拐点的处理

中心差分要求损失在参数邻域内光滑，而 ReLU 在 $z = 0$ 处不可导。若某隐藏单元的预激活值 Z_l 恰在 0 附近，参数扰动 $\varepsilon = 10^{-4}$ 后该样本的 ReLU 开关会翻转，差商算出的将是跨越折角的平均斜率而非真实梯度，导致虚假的巨大误差。

因此抽样前先计算每个隐藏单元在所有样本上的 $\min|Z_l|$ ，只保留大于阈值 10^{-2} 的「安全单元」。阈值取 10^{-2} 的依据是：扰动 ε 引起 Z_l 的变化量约为 $\varepsilon \cdot |X| \approx 10^{-4}$ （输入已归一化，量级为 1），取 10^{-2} 留出 100 倍余量。

实测在 256 个隐藏单元中有 86 个被排除。该比例合理：初始时 $b_l = 0$ ， $Z_l = XW_l$ 关于 0 对称分布，必然有相当一部分落在 0 附近。

4.3 梯度检查结果

表 4 中心差分梯度检查结果

参数	最大相对误差	是否通过 (< 1e-6)
w_1	2.72×10^{-9}	通过
b_1	4.83×10^{-9}	通过
w_2	5.92×10^{-9}	通过
b_2	2.31×10^{-8}	通过
总体	2.31×10^{-8}	通过

4.4 结果分析

四组参数的相对误差均在 10^{-8} 至 10^{-9} 量级，远低于 10^{-6} 的判据，说明手写反向传播的解析梯度与数值梯度一致，实现正确。

更值得说明的是误差的量级本身。3.3 节指出，中心差分的误差由截断误差（随 h 减小而减小）与舍入误差（随 h 减小而增大）共同决定，存在最优步长。为验证这一分析，在同一固定 mini-batch、同一批参数、同一批安全隐藏单元上，只把步长 h 从 10^{-3} 扫到 10^{-8} ，实测结果如下：

表 5 数值梯度误差随差分步长 h 的变化（实测）

步长 h	最大绝对误差	最大相对误差	主导因素
10^{-3}	1.18×10^{-7}	2.44×10^{-6}	截断误差
10^{-4} （本报告采用）	1.29×10^{-9}	2.31×10^{-8}	截断误差
3×10^{-5}	2.27×10^{-10}	6.59×10^{-9}	两者相当（最优）
3×10^{-6}	2.61×10^{-10}	9.06×10^{-8}	舍入误差
10^{-7}	5.11×10^{-9}	4.44×10^{-6}	舍入误差
10^{-8}	3.38×10^{-8}	2.27×10^{-5}	舍入误差

表中呈现出清晰的 V 形，与 3.3 节的分析逐项吻合：

- 左侧下降段服从 $O(h^2)$ 。从 10^{-3} 到 10^{-4} ，步长缩小 10 倍，误差从 1.18×10^{-7} 降到 1.29×10^{-9} ，缩小约 92 倍，与 h^2 预测的 100 倍基本一致——这一区间由截断误差主导。
- 右侧上升段服从 $O(1/h)$ 。越过 3×10^{-5} 后误差转而增大；到 10^{-7} 时实测 5.11×10^{-9} ，而由 3.3 节的舍入误差公式 $\varepsilon/L/(2h)$ 算得 4.35×10^{-9} ，两者吻合——这一区间由舍入误差主导。
- 最优步长与理论预测同量级。实测误差在 $h = 3 \times 10^{-5}$ 处最小（ 2.27×10^{-10} ），而 3.3 节由 $h^* \approx (3\varepsilon/L/|f'''|)^{1/3}$ 预测 1.4×10^{-5} 。

本报告采用的 $h = 10^{-4}$ 位于 V 形左侧的平坦段，绝对误差 1.29×10^{-9} ，仅比最优值高约 6 倍，且远离右侧舍入误差急剧上升的区域，是稳妥的取值。

结论：实测最大相对误差 2.31×10^{-8} 与该步长下中心差分的理论精度完全一致，且误差随步长的变化规律（左侧 h^2 、右侧 $1/h$ ）也得到验证。这说明残差来自差分近似本身，而非反向传播的实现错误——这正是梯度检查希望看到的结果。

五、实验二：线性 Softmax 基线

5.1 训练过程

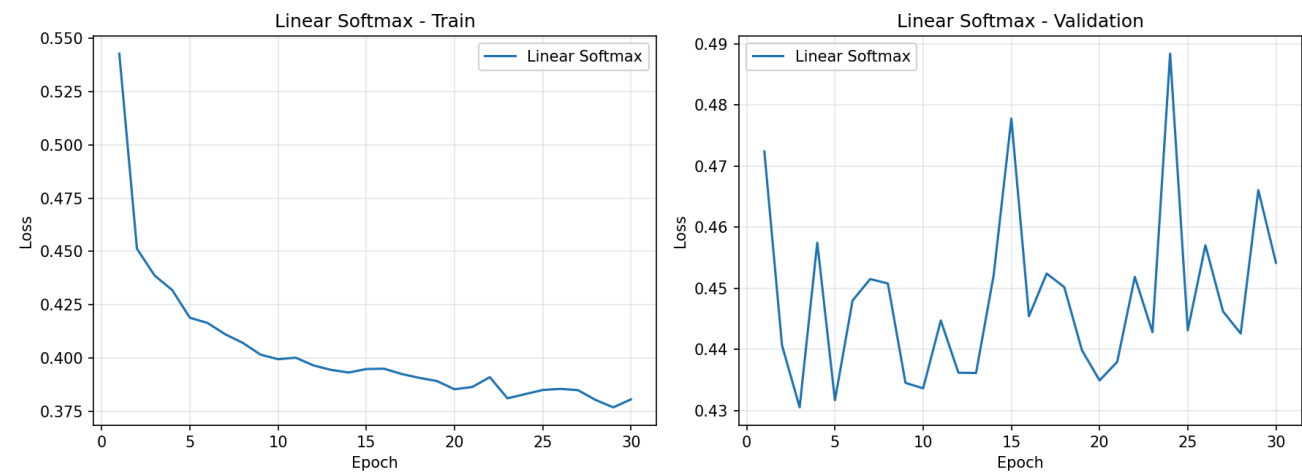


图 1 线性 Softmax 的训练与验证损失曲线

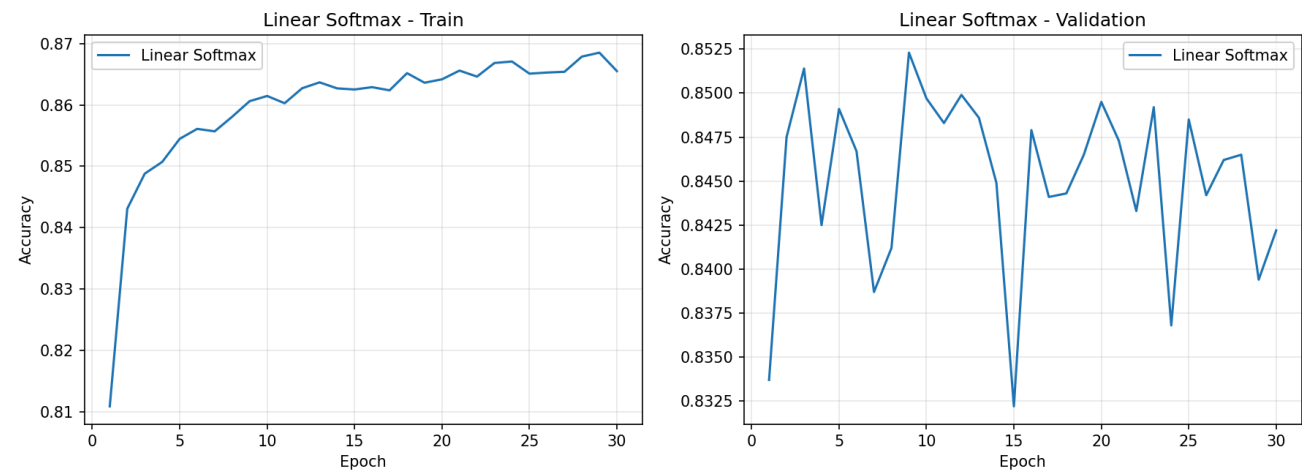


图 2 线性 Softmax 的训练与验证准确率曲线

由图 1、图 2 可见，训练在第 10 轮左右即已收敛：此后训练损失稳定在 0.38 附近不再下降，训练准确率停在 0.866，验证准确率在 0.84 上下小幅波动。末轮训练准确率 0.8655 与验证准确率 0.8422 仅相差约 2.3 个百分点，说明模型处于欠拟合而非过拟合状态——继续训练或增加轮数都不会带来改善，瓶颈在于线性模型的表达能力本身。

5.2 测试集性能与混淆矩阵

测试集 10 000 张上的结果为：损失 0.4738，准确率 0.8319，宏平均召回率 0.8319（测试集类别均衡，两者一致）。

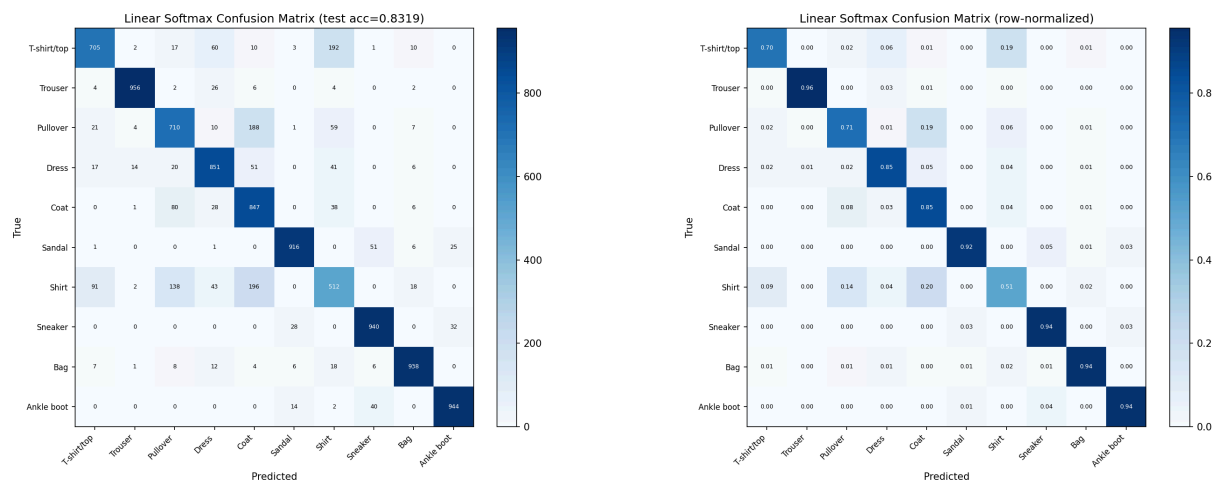


图 3 线性 Softmax 的混淆矩阵（左：样本计数；右：按行归一化）

混淆矩阵显示错误并非均匀分布，而是高度集中在上衣四类之间。最严重的四组混淆为：

表 6 线性 Softmax 最严重的混淆组合（测试集）

真实类别	预测类别	样本数
Shirt	Coat	196
T-shirt/top	Shirt	192
Pullover	Coat	188
Shirt	Pullover	138

这四组全部落在 T-shirt/top、Shirt、Pullover、Coat 之间。该现象有明确的物理原因：这四类都是上装，在 28×28 的灰度图中仅有轮廓与纹理的细微差别（领口形状、袖子有无、衣长），而线性模型只能学习「每个像素位置的权重」，无法表达「有袖子」「有领子」这类局部结构特征，因此难以区分。

5.3 逐类召回率

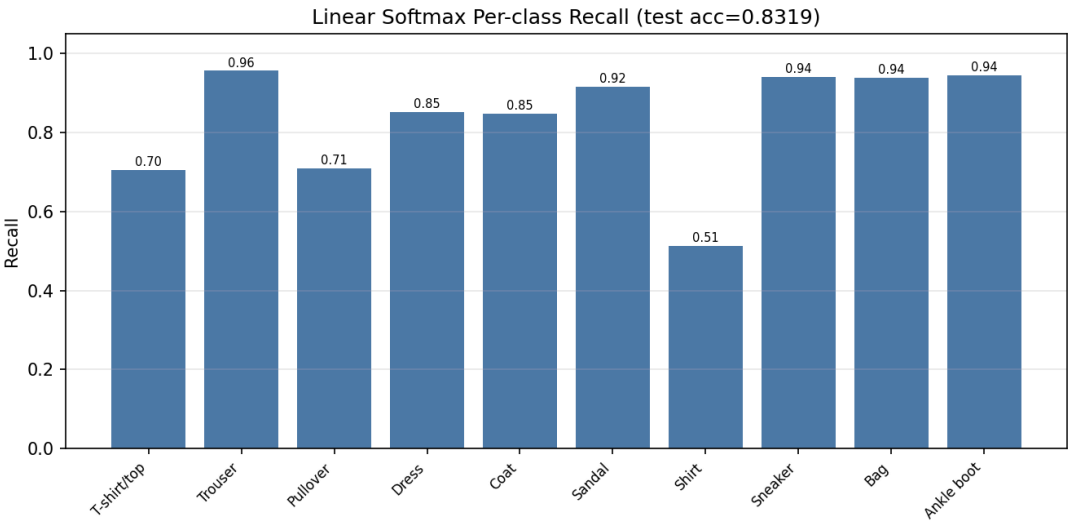


图 4 线性 Softmax 的逐类召回率

表 7 线性 Softmax 逐类召回率（测试集，按召回率升序）

类别	召回率	正确 / 总数
Shirt	0.512	512 / 1000
T-shirt/top	0.705	705 / 1000
Pullover	0.710	710 / 1000
Coat	0.847	847 / 1000
Dress	0.851	851 / 1000
Sandal	0.916	916 / 1000
Bag	0.938	938 / 1000
Sneaker	0.940	940 / 1000
Ankle boot	0.944	944 / 1000
Trouser	0.956	956 / 1000

召回率呈现明显的两极分化：下半部分的五类（Sandal 至 Trouser）都在 0.92 以上，因为这些类别的轮廓与其余类别差异显著；而上衣四类的召回率均低于 0.86，其中 Shirt 仅 0.512，即近一半的衬衫被误判。总准确率 0.8319 掩盖了这一严重的不均衡，只有逐类指标才能暴露出来。

5.4 典型错误可视化

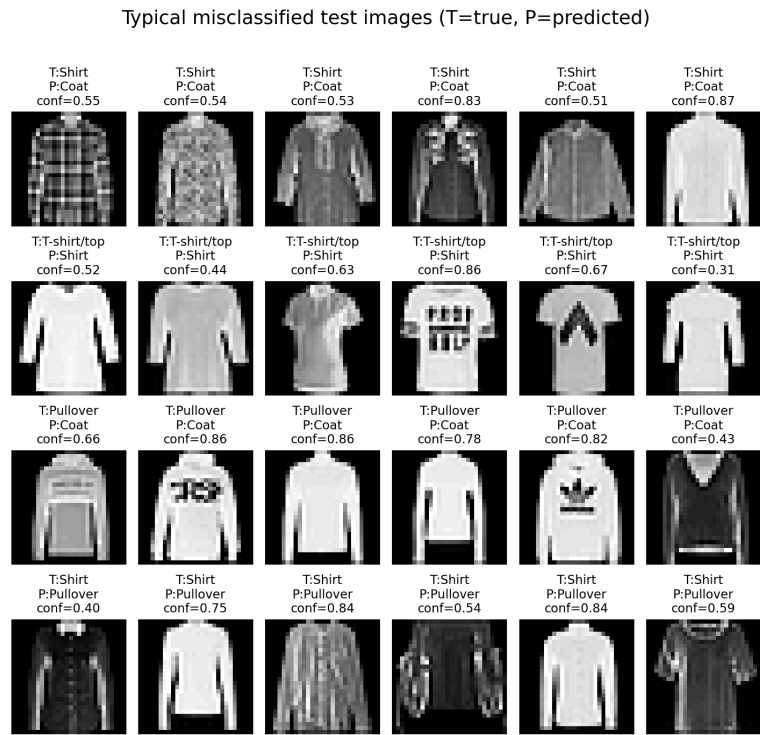


图 5 线性 Softmax 的典型错误样本 (T = 真实标签, P = 预测标签, conf = 预测置信度)

图 5 按混淆组合分组展示被误判的测试样本。可以观察到三个现象：

- 错误样本本身具有歧义性。如第二行第三列的真实标签为 Shirt、被预测为 Pullover 的样本，其轮廓确实介于衬衫与套头衫之间；第四行第一列的真实 Shirt 被预测为 Coat，图像暗且纹理复杂，肉眼也难以确定类别。这类错误反映了数据集本身的标注难度，而非模型的明显缺陷。
- 置信度普遍不高。多数错误样本的预测置信度在 0.4 - 0.6 之间，说明模型对这些样本本身也没有把握，符合「难样本」的预期。
- 存在高置信度错误。部分错误样本的置信度达到 0.84 - 0.87，这类样本才是真正值得警惕的错误——模型「自信地答错」，通常对应训练数据中覆盖不足的形态。

六、实验三：三模型受控比较

6.1 受控条件的设定

本实验的核心是「受控」：三个模型除网络结构外，其余条件全部相同。具体包括：

- 同一份数据划分：三者读取同一份落盘的训练/验证索引；
- 同一组归一化参数：训练集均值与标准差；
- 同一优化器与超参：SGD(momentum = 0.9, lr = 0.01)，批大小 128，训练 30 轮，权重衰减 0；
- 同一个随机种子，且建模前重置；

- 相同的批次顺序：每个模型开始训练前将 DataLoader 的随机数生成器拨回种子位置。若不重置，模型②看到的批次顺序会接着模型①继续抽取，三者的训练数据顺序就不再一致；
- 模型②③的隐藏层宽度相同（256），二者的参数量因此完全相同。

此外，模型②③均不使用 BatchNorm 或 Dropout，避免引入额外变量。

6.2 三模型性能表

表 8 三个模型的性能对比（隐藏层宽度 256，测试集 10 000 张）

模型	参数量	末轮训练损失	末轮验证损失	最优验证准确率	测试损失	测试准确率	训练耗时（s）
① 线性 Softmax	7 850	0.3806	0.4542	0.8523	0.4738	0.8319	24.8
② 双层 线性 （无激 活）	203 530	0.3707	0.4605	0.8572	0.4794	0.8305	35.9
③ 双层 线性 + ReLU	203 530	0.1192	0.3373	0.8965	0.3698	0.8820	36.9

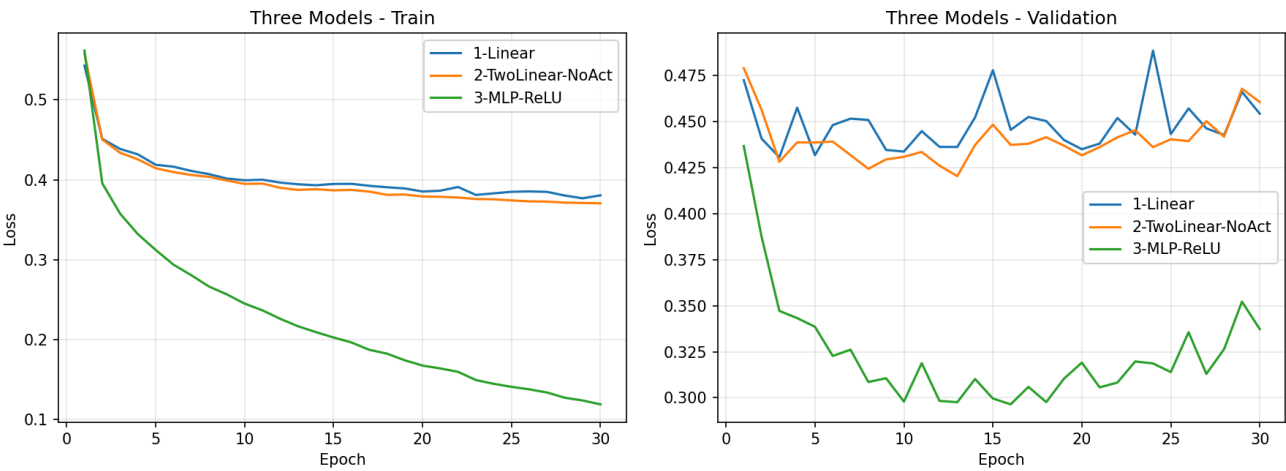


图 6 三个模型的训练与验证损失曲线

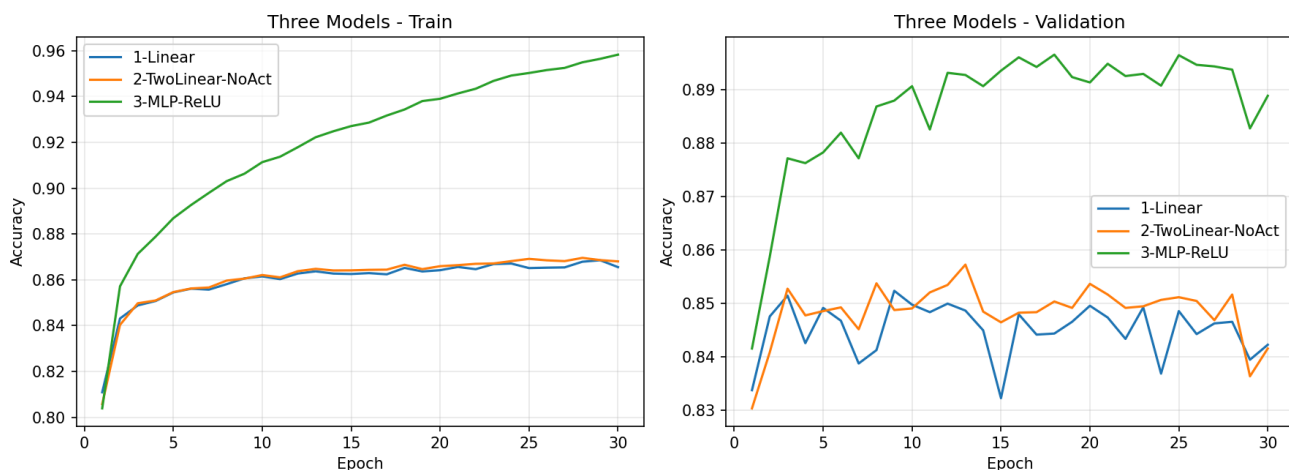


图 7 三个模型的训练与验证准确率曲线

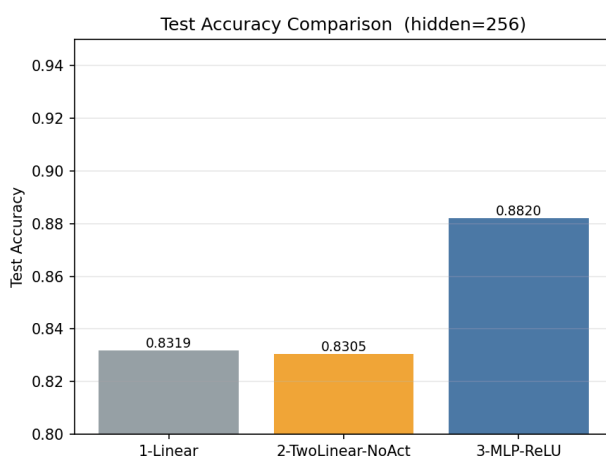


图 8 三个模型的测试准确率对比

6.3 模型②与模型①：参数量的无效增长

模型②的参数量是模型①的 25.9 倍（203 530 对 7 850），而测试准确率反而低 0.14 个百分点（0.8305 对 0.8319）。从图 6、图 7 可以看到，两条曲线几乎完全重合。

这一结果完全符合理论预期。模型②的前向传播为

$$f(x) = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) \quad (9)$$

令 $W' = W_2W_1$ 、 $b' = W_2b_1 + b_2$ ，则模型②在数学上就是一个线性分类器，其假设空间与模型①完全相同。多出的 195 680 个参数只是把同一个线性映射做了冗余参数化，不带来任何表达能力。因此

结论一：在没有非线性激活的情况下，堆叠线性层不会增加模型容量。「加深网络」的前提是层与层之间存在非线性变换，否则整个网络坍缩为一个线性模型，增加层数只增加参数与计算量，不增加表达能力。

模型②的准确率比模型①低 0.14 个百分点，处于随机波动量级，不构成显著差异。其损失曲线略高，可解释为冗余参数化使优化曲面在参数空间中变得更复杂（同样的函数对应无穷多组参数），优化效率略有下降，但这属于次要效应。

6.4 模型③与模型②：非线性的增益

模型③与模型②的参数量完全相同（均为 203 530），唯一差别是在两层线性之间插入了 ReLU。测试准确率从 0.8305 提升到 0.8820，提升 5.15 个百分点。

结论二：在参数量严格不变的条件下，仅引入一个 ReLU 非线性就带来 5.15 个百分点的提升。这组对照把「模型容量的贡献」与「非线性表达的贡献」彻底分离开：提升完全来自非线性变换带来的表达能力，而非参数量的增加。

从图 7 还可以观察到模型③出现了明显的过拟合：训练准确率持续攀升至 0.9583，而验证准确率在第 20 轮前后达到峰值 0.8965 后开始回落，末轮降至 0.8888，二者差距约 7 个百分点。对应地，验证损失从第 20 轮的 0.2964 回升至末轮的 0.3373。相比之下，模型①②的训练与验证准确率差距仅约 2.3 个百分点。这说明模型③的容量已经超出该数据规模所需，若要进一步提升泛化性能，可考虑引入权重衰减、Dropout 或早停策略。

6.5 逐类召回率对比

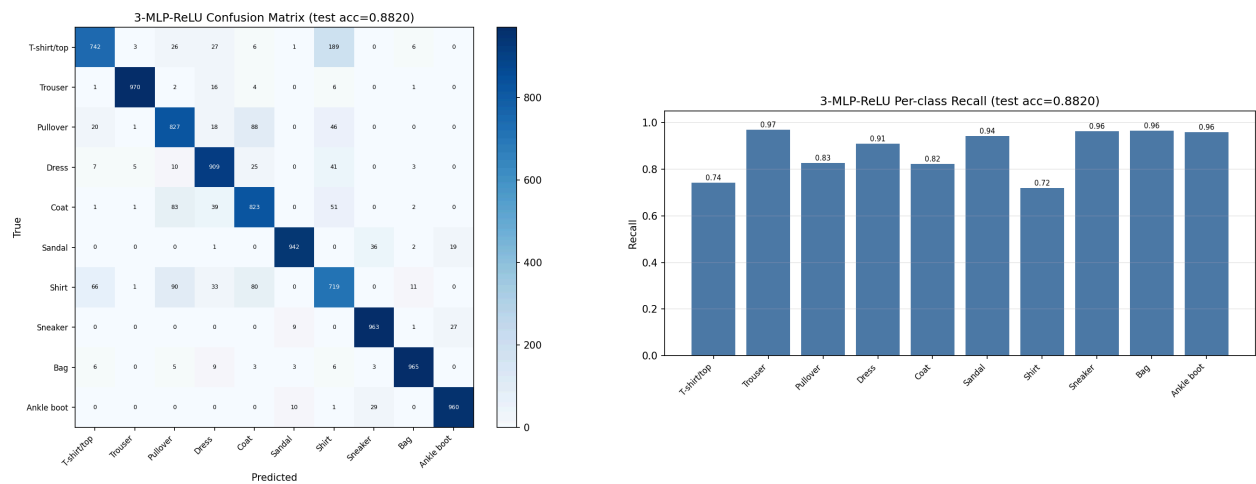


图 9 模型③（MLP + ReLU）的混淆矩阵与逐类召回率

表 9 模型①与模型③的逐类召回率对比

类别	① 线性	③ MLP+ReLU	变化
Shirt	0.512	0.719	+20.7
Pullover	0.710	0.827	+11.7
Dress	0.851	0.909	+5.8
T-shirt/top	0.705	0.742	+3.7
Trouser	0.956	0.970	+1.4
Sandal	0.916	0.942	+2.6
Bag	0.938	0.965	+2.7
Sneaker	0.940	0.963	+2.3
Ankle boot	0.944	0.960	+1.6
Coat	0.847	0.823	-2.4

（表中变化单位为百分点。）总准确率提升 5.01 个百分点，但这一提升并非均匀分摊：Shirt 一类就贡献了 20.7 个百分点的召回率提升，Pullover 提升 11.7 个百分点，而 Coat 反而下降 2.4 个百分点。这说明 ReLU 带来的增益主要作用于原本最难区分的类别——MLP 能够通过隐藏层学习到「有无袖子」「领口形状」这类局部结构特征，而线性模型只能学习全局的像素权重。

值得注意的是 Coat 的召回率略有下降，且从混淆矩阵可见 Coat 有 83 例被误判为 Pullover。这表明非线性的增益在不同类别间的分配并不均衡，部分形态相近的类别仍存在此消彼长的关系。

七、实验四：训练诊断——学习率

7.1 现象

在调试 MLP 训练时出现如下现象：当学习率取 1.0 时，训练损失不下降反而剧烈上升，模型在整个训练过程中完全没有学到任何东西，验证准确率始终停留在随机水平（10%）附近。

7.2 原因判断

初步判断为步长超过了损失曲面的稳定范围。理由如下：

SGD 的参数更新量为 $\Delta\theta = lr \cdot g$ 。当学习率过大时，参数一步就跨过了损失曲面的谷底，导致损失不减反增；损失增大后梯度进一步增大，形成正反馈，损失量级持续攀升直至数值溢出。

动量项会加剧这一过程。带动量的 SGD 在梯度方向稳定时，等效步长约为 $lr / (1 - momentum)$ 。本实验取 $momentum = 0.9$ ，等效放大倍数约 10 倍，即 $lr = 0.1$ 配 $momentum = 0.9$ 的有效步长已相当于朴素 SGD 的 $lr = 1.0$ 。这解释了为何在带动量的设定下，学习率需要取得比通常更小。

需要强调的是，上述判断只是假设。可能导致训练失败的因素还包括初始化不当、数据未归一化、损失函数实现错误等。必须通过控制变量的实验来排除其他可能。

7.3 验证方法与结果

验证策略：固定网络结构、数据划分、优化器类型、动量系数、批大小、训练轮数与随机种子全部不变，只改变学习率这一个因素，取 $lr \in \{0.01, 0.1, 1.0\}$ ，训练 20 轮。由于其他所有条件都被固定，学习率是唯一的自变量，实验结果可以归因。

表 10 不同学习率下的训练结果（其余条件完全一致）

学习率	首个 epoch 损失	末轮训练损失	最优验证准确率	结论
0.01	0.5616	0.1677	0.8965	正常收敛
0.1	0.5556	0.2346	0.8753	收敛但明显更差
1.0	69.9874	2.3081	0.1374	未收敛 / 震荡

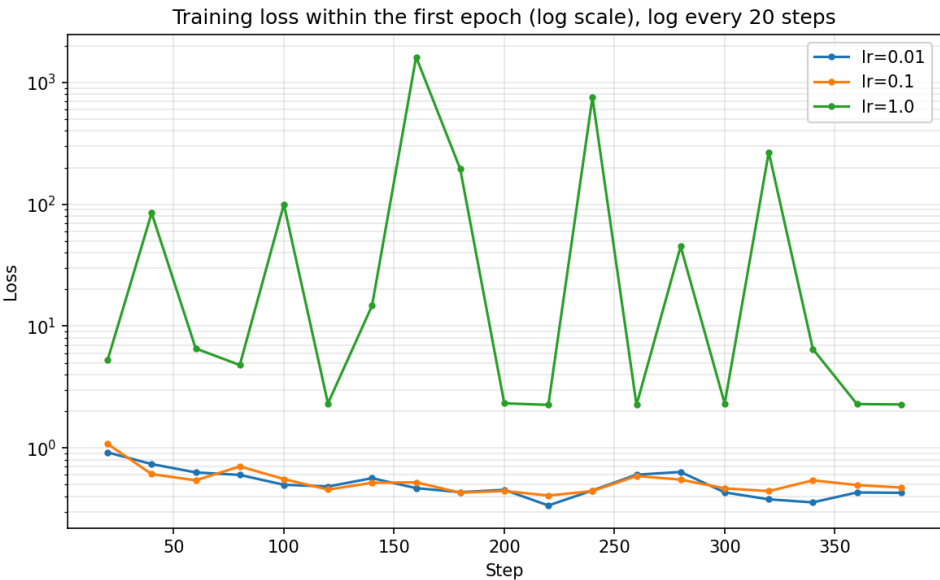


图 10 第一个 epoch 内逐 step 的训练损失（横轴为迭代步数，纵轴取对数刻度）

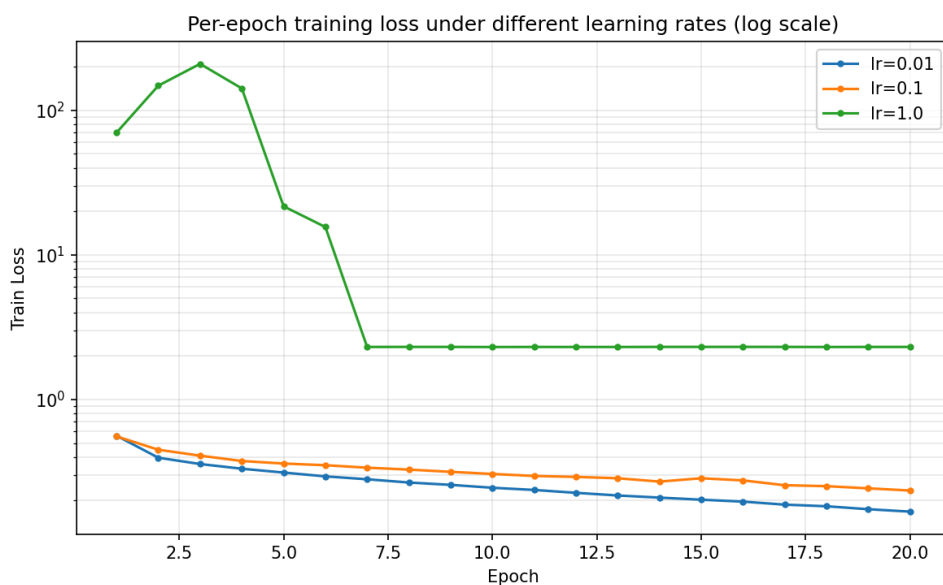


图 11 不同学习率下逐 epoch 的训练损失（纵轴取对数刻度）

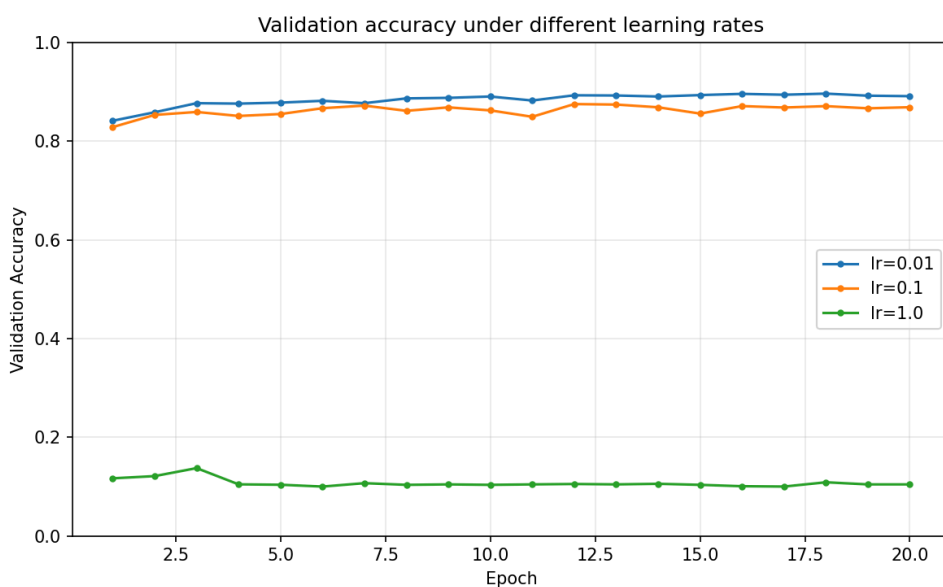


图 12 不同学习率下的验证准确率

7.4 结果分析

学习率 0.01：损失平稳单调下降，末轮训练损失 0.1677，验证准确率 0.8965，训练正常。

学习率 0.1：仍能收敛，但末轮训练损失 0.2346（比 0.01 高 40%），验证准确率 0.8753，低 2.1 个百分点。说明学习率偏大时的退化是渐进的，而非突变的——在发散之前，性能就已经在悄悄损失。

学习率 1.0：训练完全失败。图 10 显示第一个 epoch 内损失就在 0.5 至 1800 之间反复震荡，毫无下降趋势；首个 epoch 的平均损失高达 69.99。图 11 显示，从第 7 个 epoch 起损失稳定停在 2.303。

这个数值极具说明性： $2.303 = \ln 10$ ，正是 10 类等概率分布下的交叉熵（ $-\ln(1/10)$ ）。它意味着模型输出的 logits 已经趋同，Softmax 退化为均匀分布，即模型对任何输入都给出完全相同的预测。图 12 中该曲线的验证准确率恰为 10%，与此完全吻合——模型已经彻底失去判别能力。

7.5 诊断结论

结论三：在其余条件全部固定的情况下，仅改变学习率就使同一个模型从正常收敛（0.8965）变为完全失败（0.1374）。这排除了网络结构、数据划分、初始化方式等其他解释，证实训练失败的原因是步长过大。定量地看，该配置下的安全学习率在 0.01 附近；0.1 已出现可测量的退化；1.0 完全失效。

作为该结论的进一步佐证，将学习率继续加大到 2.0 时，损失会直接溢出到 10^{26} 量级，与「步长越大发散越快」的机理一致。

八、讨论

8.1 实验的可复现性

为保证结果可复现，实现中处理了三个容易被忽略的问题：

1. 数据划分落盘。训练/验证索引保存为文件，四个实验共用。若每次运行重新随机划分，三个模型的训练集与验证集将互不相同，性能差异无法归因。
2. 建模前重置随机种子。实验中发现：加载数据时遍历 DataLoader 会消耗全局的 PyTorch 随机源，导致「脚本开头设一次种子」的写法在不同脚本中产生不同的模型初始化。同一个线性模型在独立运行时得到 0.8329，而在比较脚本中得到 0.8319。统一为「每个模型构建前重置种子」后，两处结果一致。这个问题直接影响「比较是否受控」，是本次实验中最值得记录的教训。
3. 重置 DataLoader 的打乱顺序。打乱顺序由 DataLoader 内部的随机数生成器决定，其状态会在多个模型间连续累积，必须显式重置。

8.2 实验的局限

- 单次运行，未做多次重复。表 8 中模型②与① 0.14 个百分点的差异、表 9 中 Coat 类 2.4 个百分点的下降，均未经过多次重复实验验证，可能处于随机波动范围内。严格结论需要多组种子下的均值与标准差。
- 未调优即比较。为保证受控，三个模型共用同一套超参数，未针对各自做学习率调优。这是「受控」与「最优」之间的取舍：本实验选择受控优先，因此三个模型的绝对性能都未达到各自的最优水平。
- 模型③已出现过拟合（训练与验证准确率相差约 7 个百分点），而实验未引入正则化或早停。这为后续改进留下了空间，但不影响本次受控比较的结论。

九、结论

1. 反向传播实现正确。纯 NumPy 实现的单隐藏层 MLP 前向与反向传播，经中心差分梯度检查验证，四组参数的最大相对误差为 2.31×10^{-8} ，远低于 10^{-6} 的判据，且该量级与 float64 下中心差分的理论精度极限一致。
2. 堆叠线性层不增加表达能力。双层无激活模型的参数量是线性模型的 25.9 倍，测试准确率却低 0.14 个百分点。数学上 W_2W_1 仍是单个线性映射，二者假设空间完全相同——「加深」必须以非线性为前提。
3. 非线性的增益是显著的且可分离的。在参数量完全相同的条件下，仅加入一个 ReLU 就使测试准确率提升 5.15 个百分点 ($0.8305 \rightarrow 0.8820$)。该提升集中在原本最难的上衣类：Shirt 的召回率从 0.512 提升到 0.719。
4. 学习率过大是训练失败的直接原因。在其余条件全部固定的前提下，学习率从 0.01 增至 1.0 使验证准确率从 0.8965 崩溃至 0.1374，损失稳定在 $\ln 10$ （即均匀随机猜测）。实验同时表明，带动量 SGD 的等效步长约为 $lr/(1-\text{momentum})$ ，因此需要更小的学习率。
5. 总准确率会掩盖类别间的不均衡。线性基线的总准确率 0.8319 看似尚可，但 Shirt 类的召回率仅 0.512。只有结合混淆矩阵与逐类召回率，才能发现错误高度集中在上衣四类之间这一关键事实。

附录 A 代码与复现说明

A.1 目录结构

fashion-mnist-lab/	
├─ config.py	全局配置：随机种子、划分比例、超参数、路径
├─ data_utils.py	数据加载、固定划分、DataLoader 构建
├─ models.py	三个 PyTorch 模型
├─ numpy_mlp.py	纯 NumPy 的 forward 与 backward
├─ grad_check.py	中心差分梯度检查
├─ trainer.py	三模型共用的训练与评估循环
├─ evaluate.py	混淆矩阵、逐类召回率、曲线、典型错误可视化
├─ train_linear.py	实验二：线性 Softmax 基线
├─ train_compare.py	实验三：三模型受控比较
├─ diagnose.py	实验四：学习率诊断
├─ run_all.py	一键跑通全部实验
└─ outputs/	所有结果：图片、CSV、JSON

A.2 运行方式

环境依赖：Python 3.13、NumPy、PyTorch(cpu)、torchvision、Matplotlib。全部实验在 CPU 上完成。

```
# 一键运行全部实验（约 4 分钟）
python run_all.py

# 也可单独运行各部分
python grad_check.py      # 梯度检查，约 5 秒
python train_linear.py    # 线性基线，约 30 秒
python train_compare.py   # 三模型比较，约 1.7 分钟
python diagnose.py        # 学习率诊断，约 1.8 分钟
```

数据集首次运行时自动下载（约 30 MB）。所有超参数集中在 `config.py` 中，修改一处即可全局生效。

A.3 随机种子说明

全局种子 `SEED = 42`，控制数据划分、参数初始化与 `DataLoader` 的打乱顺序。由于数据划分索引已落盘（`outputs/split_indices.npz`），删除该文件将重新划分数据集，数值结果随之改变，但三个模型仍共用同一份划分，比较依然受控。在相同环境下重复运行，结果完全一致。

A.4 提交物清单

表 11 提交物与作业要求的对应关系

作业要求	对应文件
梯度检查结果	表 4、5； <code>outputs/grad_check.csv</code>
三模型性能表	表 8； <code>outputs/compare_table.csv</code>
训练与验证曲线	图 1、2、6、7、11、12
混淆矩阵	图 3、9
逐类召回率	表 7、9；图 4
典型错误	图 5
可运行源代码	<code>fashion-mnist-lab/</code> 全部代码
README 说明文件	<code>fashion-mnist-lab/README.md</code>